

Queue

# Module 1: Philosophy & Foundations of Queue

# The Origin of Queue & FIFO Principle

- **Core Concept** Queues were created to manage "**order of arrival**" in digital systems, mirroring real-world human order.
- **FIFO Principle** (First-In, First-Out) A structural guarantee of fairness—the first item created is always the first to be serviced and completed.
- Real-World **Examples** Supermarket **checkout** lines, factory assembly lines, call center customer waiting lines.

# Core Operations & Elements

- **Head & Tail**

- Head The **single point** where items are **retrieved/dequeued**.

- Tail The **single point** where new items are **added/enqueued**.

- **Enqueue** (Put): Pushing new data onto the Queue at the Tail.

- **Dequeue** (Get): Retrieving the oldest data from the Head for processing.

- **O(1) Speed Guarantee** The strict rule of Queue—whether storing 10 items or 1,000,000 items, Enqueue/Dequeue **execution time remains constant**.

# Internal Memory Layout

- Circular Buffer (**Fixed-Size** Allocation)
  - **Pre-allocates** a **fixed** block of memory and **rotates** Head/Tail **pointers** in a circular loop like a **round dining table with fixed chairs**. When an item dequeues from the Head, that slot becomes immediately available for new enqueues without moving existing items
  - Head & Tail Pointers. **Track** next retrieval slot and next insert slot.
  - Modulo Arithmetic (%): Used to wrap indices around  **$(\text{Tail} + 1) \% N$**

Circular Buffer (Capacity = 6)

# Internal Memory Layout

- **Linked List (Dynamic Chain Allocation)**
  - Allocates memory **on-demand per incoming item**, connecting elements via pointers (like adding dynamic cars to a train).
  - Node Structure: Contains **Data** + **Next Pointer** (RAM address of the next node).
  - **Unbounded capability** — grows infinitely until system RAM is exhausted
  - Garbage Collection Overhead: Frequent allocation/deallocation increases OS/GC workload.

# Queue using Linked List

Status: Initializing Empty Queue



# System Decoupling

- **Tight** Coupling Problem
  - System A directly calls System B. If System B drops or slows down, System A crashes instantly.
- Queue as a **Middleman**
  - Separates **Producers** from **Consumers** so **neither needs** internal knowledge of the other, communicating solely via the Queue.

# Buffer Mechanism for Traffic Spikes

- **Spike Traffic**

- Sudden **bursts** of incoming data/users (e.g., high-demand concert ticket drops).

- **Shock Absorption**

- Acts as a reservoir holding incoming bursts, protecting backend services from being overwhelmed.

# Load Leveling

- **Smoothing Demand**
  - Converts irregular, volatile traffic **spikes** into a **smooth**, steady processing flow.
- Pull-Based Pattern
  - Consumers pull tasks from the Queue at their own operational capacity rather than having tasks forced upon them (Push-based).

# Module 2: Real-World Processing Patterns

# Producer-Consumer Pattern

- **Producer** Role:
  - Task generator (e.g., checkout requests, web crawlers, IoT sensor logs).
- **Consumer** Role:
  - Task executor (e.g., payment gateways, web page downloaders, analytics engines).
- **Interactions:**
  - The Queue serves as the neutral handoff boundary between producers and consumers.

# Without Queue: Fast Producer vs Slow Consumer

Status: Direct Push-based system (No Queue)

# Handling Speed Mismatches

- The Reality
  - Producers almost always **outperform** Consumers (e.g., Producing 1,000 tasks/sec vs. Consuming 100 tasks/sec).
- Queue Solution
  - Enables Producers **to continue generating without blocking**, preventing Consumers from getting drowned by incoming volume.

# With Queue: Buffer Absorbs Producer Spikes

Status: Queue added (Pull-based architecture)



# Horizontal Scaling via Multiple Consumers

- Competing Consumers Pattern
  - **Distributes** heavy Queue workloads across **multiple parallel** Consumer instances.
- **Parallel Processing:**
  - Tasks are evenly distributed across available Consumers, accelerating throughput linearly.

# Blocking Queue (Thread-Based)

- "Wait Until Available" Concept
  - When the Queue is **empty**, the Consumer thread **blocks** (sleeps/waits) until data arrives.
- Trade-offs
  - Simple and thread-safe, but consumes thread resources during idle wait states

# Blocking Queue Behavior

Status: Queue is empty

## Blocking Queue: Queue Full Case (Producer Blocked)

Status: Bounded Queue (Capacity = 3)

# Non-Blocking / Event-Driven Queue (Async)

- **"Yield and Switch"** Concept
  - When **empty**, the Consumer registers an interest, **yields** execution control, and processes **other tasks** instead.
- Event Loop Notification
  - System notifies Consumer upon new Enqueue events, maximizing CPU resource efficiency.

# Bounded Queues & Overflow Control

- Unbounded Queue Risk
  - Infinite capacity risks total memory consumption and system-wide crashes.
- Bounded Queue Safeguard
  - Sets strict maximum thresholds (e.g., limit of 5,000 items) for structural safety.

## Async Queue: Empty Queue Case (Non-Blocking / Yield Control)

Status: Event Loop managing Coroutines

## Async Queue: Queue Full Case (Producer Yields Control)

Status: Bounded async Queue (maxsize = 2)



# Module 3: Structural Comparisons

# Queue vs. Stack

- Queue (FIFO)
  - Like a retail checkout line—the first item added at the Tail is the first processed at the Head.
- Stack (LIFO)
  - Like a stack of plates—the last item added at the top is the first item removed

# Data Structures: Queue vs Stack



# Queue vs. Array

- Queue
  - Restricted Access—Strictly forces **Enqueue** at Tail and **Dequeue** at Head; **random midpoint access is restricted.**
- Array
  - Random Access—Allows instant access/modification at any index without removing prior elements.

# Data Structures: Array vs Queue



# Queue vs. Linked List

- Linked List
  - A **low-level data structure** / underlying memory implementation.
- Queue
  - An abstract data type (ADT) / **behavioral pattern**.
- Relationship
  - Linked Lists are used inside Queues to create dynamic, unbounded queues while preserving  $O(1)$  time complexity for both Enqueue and Dequeue operations.

# Data Structures: Queue vs Linked List



# Module 4: Asyncio.Queue



# Fundamentals of Asyncio & Asynchronous Concepts

- Traditional **execution** blocks the CPU while **waiting for I/O** operations
- **Yields** execution control instantly during I/O **waits**, allowing **other tasks** to run without the heavy overhead of spawning OS-level threads
- when Task A hits a waiting state, the Event Loop pauses Task A and shifts processing power to Task B immediately.

# 01\_synchronous\_vs\_asynchronous.py

- เปรียบเทียบให้เห็นชัดเจนว่า Synchronous ทำให้ CPU นั่งรออย่างไร และ Asynchronous สลับงานไปทำทันทีได้อย่างไร

# Chapter 1: Sync vs Async Execution



# Understanding asyncio.Queue

- **Async Communication Bridge** A specialized data structure designed for passing messages or tasks between multiple Coroutines (async functions) running on a single Event Loop
- **Concurrency Safety** Built specifically to **handle concurrent reads and writes** across Coroutines safely **without requiring manual locks** or **synchronization primitives**
- **Strict FIFO Guarantee** Preserves standard Queue mechanics where the first item pushed into the queue is guaranteed to be the first item popped out.

## 02\_basic\_asyncio\_queue.py

- การใช้ `asyncio.Queue` เชื่อม Coroutine ฝั่งผู้ผลิต (Producer) และผู้บริโภค (Consumer) เข้าด้วยกัน อย่างปลอดภัยแบบ FIFO

## Chapter 2: asyncio.Queue Basics (FIFO)

# Basic Operations (Put & Get)

- **Enqueuing (Put)** The **producer Coroutine pushes** data into the queue. If space is available, insertion completes immediately.
- **Dequeuing (Get)** The **consumer Coroutine retrieves** data from the head of the queue. If data is present, it returns the item instantly.
- **Non-blocking Await Mechanics** If the queue is empty, a `get()` operation pauses the calling Coroutine and yields control back to the Event Loop instead of freezing the whole program.

# 03\_put\_and\_get\_mechanism.py

- พฤติกรรมเมื่อ คิวว่างเปล่า (Empty Queue) โดยคำสั่ง `queue.get()` จะสั่งหยุดรอ (Await) โดยอัตโนมัติ ไม่ทำให้ Event Loop ค้าง และสลับไปรัน Coroutine อื่นได้



## Chapter 3: Non-Blocking Awaitable Get

# Capacity Control & Resource Management (Bounded Queues)

- **Setting Capacity Limits (maxsize)** Allows developers to cap the maximum number of items the queue can hold, preventing memory exhaustion under heavy traffic.
- **Queue Full Behavior** When reaching capacity, a `put()` operation automatically pauses (awaits) until a consumer removes an item to free up space.
- **Native Backpressure** Throttles fast Producers automatically so their output **speed matches** the throughput rate of downstream Consumers.

# 04\_bounded\_queue\_backpressure.py

- กลไก Backpressure โดยกำหนด maxsize เพื่อชะลอฝั่ง Producer เมื่อคิวเต็ม ป้องกัน memory exhaustion

## Chapter 4: Bounded Queue (maxsize=2)

# Task Completion & Lifecycle Tracking

- **Work-in-Progress Tracking** Maintains an internal counter of tasks that have been retrieved from the queue but are still being processed.
- **Completing a Task (task\_done)** Consumers must notify the queue whenever a retrieved item finishes processing to decrement the internal active task counter.
- **Awaiting Queue Clearance (join)** Blocks execution flow until every item added to the queue has been processed and acknowledged via `task_done()`, ensuring clean shutdown sequences.

# 05\_task\_completion\_and\_join.py

- วิธีเขียนระบบประมวลผลแบบสง่างาม (Graceful Shutdown) โดยใช้ `task_done()` ร่วมกับ `queue.join()` เพื่อการันตีว่างานทุกชิ้นถูกเคลียร์ครบก่อนปิดโปรแกรม

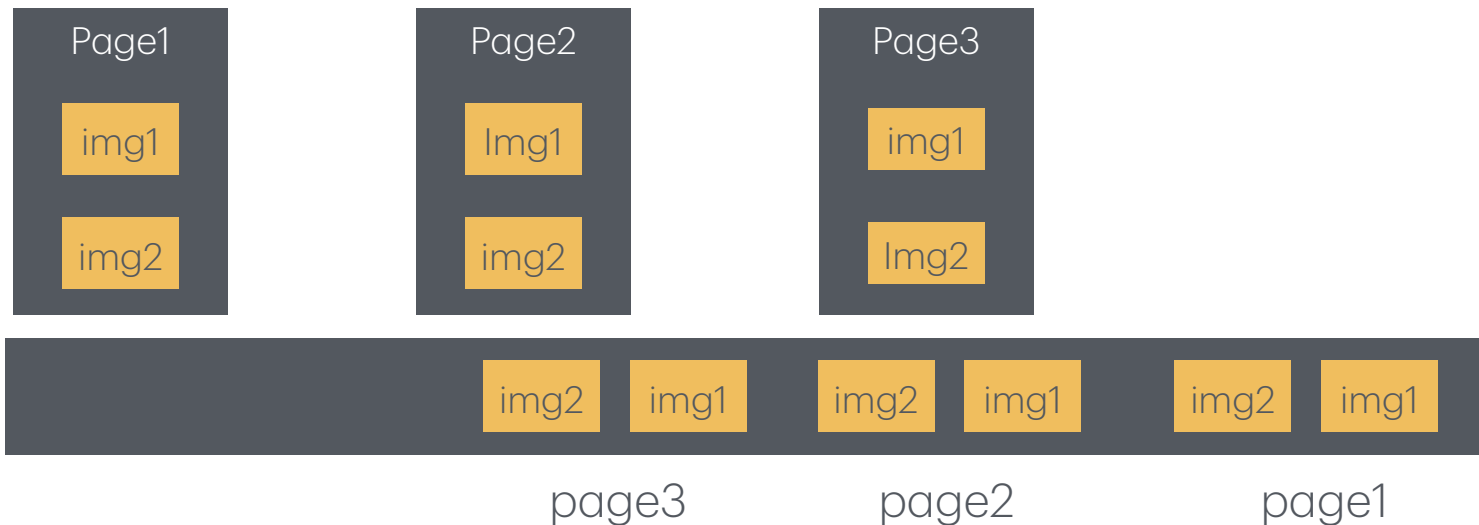
## Chapter 5: `task_done()` & `queue.join()`

# Web Scraper & Image Download



## Chapter 6: Web Scraper & Downloader (1 Producer, 1 Consumer)

# Web Scraper & Image Download



```
[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สดารถเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https://example.com/images/page_1_img1.jpg
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https://example.com/images/page_1_img2.jpg
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https://example.com/images/page_2_img1.jpg
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https://example.com/images/page_2_img2.jpg
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https://example.com/images/page_3_img1.jpg
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https://example.com/images/page_3_img2.jpg
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด 6 รูป
```

```
43  async def main():
44      pages = ["page_1", "page_2", "page_3"]
45      queue = asyncio.Queue()
46
47      # 1. สร้าง Task สำหรับ Producer และ Consumer (1 ตัว)
48      producer_task = asyncio.create_task(link_scraper(queue, pages))
49      downloader_task = asyncio.create_task(image_downloader(queue, "Downloader_01"))
50
51      # 2. รอให้ Producer เสร็จสิ้นงาน
52      await producer_task
53
54      # 3. รอให้ Consumer เสร็จสิ้นงานใน Queue ทั้งหมด
55      await queue.join()
56
57      # 4. ส่ง None 1 ครั้ง เพื่อแจ้งให้ Downloader ตัวเดี๋ยวนี้อยุดทำงาน
58      await queue.put(None)
59      await downloader_task
60
61  if __name__ == "__main__":
62      asyncio.run(main())
```

```
[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สตาร์ทเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https:
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https:
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https:
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https:
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https:
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https:
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด
```

```
43 async def main():
44     pages = ["page_1", "page_2", "page_3"]
45     queue = asyncio.Queue()
46
47     # 1. สร้าง Task สำหรับ Producer และ Consumer (1 ตัว)
48     producer_task = asyncio.create_task(link_scraper(queue, pages))
49     downloader_task = asyncio.create_task(image_downloader(queue, "Downloader_01"))
50
51     # 2. รอให้ Producer หาลิงก์จนครบ
52     await producer_task
53
54     # 3. รอให้ Consumer เคลียร์งานใน Queue จนหมด
55     await queue.join()
56
57     # 4. ส่ง None 1 ครั้ง เพื่อแจ้งให้ Downloader ตัวเดี๋ยวนี้นหยุดทำงาน
58     await queue.put(None)
59     await downloader_task
60
61 if __name__ == "__main__":
62     asyncio.run(main())
```

```

[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สตาร์ทเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https://
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https://
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https://
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด

```

```

3  async def link_scraper(queue: asyncio.Queue, page_urls: list):
4      """Producer: สแกนหาลิงก์รูปภาพแล้วใส่ลง Queue"""
5      print("[Producer] เริ่มสแกนหาลิงก์รูปภาพ...")
6
7      for page in page_urls:
8          print(f" -- [Producer] สแกนหน้าเว็บ: {page}")
9          await asyncio.sleep(0.3) # จำลองเวลาอ่าน HTML
10
11         # เจอลิงก์รูปภาพ 2 รูปต่อ 1 หน้า
12         img_url_1 = f"https://example.com/images/{page}_img1.jpg"
13         img_url_2 = f"https://example.com/images/{page}_img2.jpg"
14
15         await queue.put(img_url_1)
16         await queue.put(img_url_2)
17
18     print("[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!\n")
19

```

```
[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สตาร์ทเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https://exam
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https://exam
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https://exam
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https://exam
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https://exam
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https://exam
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด 6 รูป
```

```
20 async def image_downloader(queue: asyncio.Queue, worker_name: str):
21     """Consumer (1 ตัว): ดึงลิงก์จาก Queue ที่ละใบมาทำการดาวน์โหลด"""
22     downloaded_count = 0
23     print(f"[{worker_name}] สตาร์ทเตรียมพร้อมโหลดรูป...")
24
25     while True:
26         # ดึงลิงก์ออกจาก Queue
27         img_url = await queue.get()
28
29         # ตรวจสอบสัญญาณหยุด (Sentinel Value)
30         if img_url is None:
31             queue.task_done()
32             break
33
34         downloaded_count += 1
35         print(f" -> [{worker_name}] (รูปที่ {downloaded_count}) กำลังโหลด: {img_url}")
36         await asyncio.sleep(0.5) # จำลองระยะเวลาดาวน์โหลดไฟล์ผ่าน Network
37
38         # แจ้ง Queue ว่าประมวลผลลิงก์นี้เรียบร้อยแล้ว
39         queue.task_done()
40
41     print(f"[{worker_name}] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด {downloaded_count} รูป")
42
```

```
[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สตาร์ทเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https://
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https://
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https://
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด
```

```
3  async def link_scraper(queue: asyncio.Queue, page_urls: li
4      """Producer: สแกนหาลิงก์รูปภาพแล้วใส่ลง Queue"""
5      print("[Producer] เริ่มสแกนหาลิงก์รูปภาพ...")
6
7      for page in page_urls:
8          print(f" -- [Producer] สแกนหน้าเว็บ: {page}")
9          await asyncio.sleep(0.3)  # จำลองเวลาอ่าน HTML
10
11         # เจอลิงก์รูปภาพ 2 รูปต่อ 1 หน้า
12         img_url_1 = f"https://example.com/images/{page}_im
13         img_url_2 = f"https://example.com/images/{page}_im
14
15         await queue.put(img_url_1)
16         await queue.put(img_url_2)
17
18         print("[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!\n")
19
```

```
[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สตาร์ทเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https://exam
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https://exam
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https://exam
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https://exam
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https://exam
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https://exam
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด 6 รูป
```

```
20 async def image_downloader(queue: asyncio.Queue, worker_name: str):
21     """Consumer (1 ตัว): ดึงลิงก์จาก Queue ที่ละใบมาทำการดาวน์โหลด"""
22     downloaded_count = 0
23     print(f"[{worker_name}] สตาร์ทเตรียมพร้อมโหลดรูป...")
24
25     while True:
26         # ดึงลิงก์ออกจาก Queue
27         img_url = await queue.get()
28
29         # ตรวจสอบสัญญาณหยุด (Sentinel Value)
30         if img_url is None:
31             queue.task_done()
32             break
33
34         downloaded_count += 1
35         print(f" -> [{worker_name}] (รูปที่ {downloaded_count}) กำลังโหลด: {img_url}")
36         await asyncio.sleep(0.5) # จำลองระยะเวลาดาวน์โหลดไฟล์ผ่าน Network
37
38         # แจ้ง Queue ว่าประมวลผลลิงก์นี้เรียบร้อยแล้ว
39         queue.task_done()
40
41     print(f"[{worker_name}] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด {downloaded_count} รูป")
42
```



```
[Producer] เริ่มสแกนหาลิงก์รูปภาพ...
-- [Producer] สแกนหน้าเว็บ: page_1
[Downloader_01] สตาร์ทเตรียมพร้อมโหลดรูป...
-- [Producer] สแกนหน้าเว็บ: page_2
-> [Downloader_01] (รูปที่ 1) กำลังโหลด: https://
-- [Producer] สแกนหน้าเว็บ: page_3
-> [Downloader_01] (รูปที่ 2) กำลังโหลด: https://
[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!

-> [Downloader_01] (รูปที่ 3) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 4) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 5) กำลังโหลด: https://
-> [Downloader_01] (รูปที่ 6) กำลังโหลด: https://
[Downloader_01] ทำงานเสร็จสิ้น! ดาวน์โหลดรวมทั้งหมด
```

```
3  async def link_scraper(queue: asyncio.Queue, page_urls: li
4      """Producer: สแกนหาลิงก์รูปภาพแล้วใส่ลง Queue"""
5      print("[Producer] เริ่มสแกนหาลิงก์รูปภาพ...")
6
7      for page in page_urls:
8          print(f" -- [Producer] สแกนหน้าเว็บ: {page}")
9          await asyncio.sleep(0.3) # จำลองเวลาอ่าน HTML
10
11         # เจอลิงก์รูปภาพ 2 รูปต่อ 1 หน้า
12         img_url_1 = f"https://example.com/images/{page}_im
13         img_url_2 = f"https://example.com/images/{page}_im
14
15         await queue.put(img_url_1)
16         await queue.put(img_url_2)
17
18         print("[Producer] สแกนหาลิงก์รูปภาพเสร็จสิ้น!\n")
19
```

# Web Scraper & Image Download

```
43 async def main():
44     pages = ["page_1", "page_2", "page_3"]
45     queue = asyncio.Queue()
46
47     # 1. สร้าง Task สำหรับ Producer และ Consumer
48     producer_task = asyncio.create_task(producer(pages, queue))
49     downloader_task = asyncio.create_task(downloader(queue))
50
51     # 2. รอให้ Producer ทำงานเสร็จ
52     await producer_task
53
54     # 3. รอให้ Consumer เคลียร์งานใน Queue
55     await queue.join()
56
57     # 4. ส่ง None 1 ครั้ง เพื่อแจ้งให้ Downloader เสร็จ
58     await queue.put(None)
59     await downloader_task
60
61 if __name__ == "__main__":
62     asyncio.run(main())
```

- Consumer และ Producer **เริ่มทำงานพร้อมกัน**
  - เมื่อส่ง create\_task() ทั้ง Producer และ Consumer จะถูกโยนขึ้น Event Loop ทันที
  - เมื่อ Producer put() รูปแรกเข้าคิว Consumer ที่รออยู่จะ get() รูปแรกนั้นไปดาวน์โหลดทันที โดยไม่ต้องรอให้ Producer สแกนครบทุกหน้า
- หน้าที่ของ await producer\_task ใน main()
  - บรรทัดนี้มีไว้เพื่อบอกโปรแกรมหลักว่า "อย่าเพิ่งข้ามไปบรรทัด queue.join() หรือส่ง None ปิดระบบนะ ให้รอจนกว่า Producer จะผลิตของครบก่อน"
  - ในระหว่างที่ main() หยุดรอที่บรรทัดนี้ ตัว Event Loop ด้านหลังยังคงรันทั้ง Producer (ผลิตของเพิ่ม) และ Consumer (ดึงของไปทำ) ขนานกันไปเรื่อยๆ